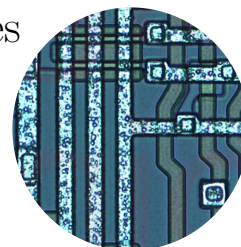


A Evolução dos Microprocessadores

© Juliano Mourão Vieira

2 de setembro de 2026



Sumário

1	O que é arquitetura de computadores?	2
1.1	ISA	2
1.2	Microarquitetura	3
1.3	Harvard vs. von Neumann	4
1.4	Três ISA's modernas	5
1.5	ISA RISC versus ISA CISC: decisões de implementação . . .	7
1.6	Modos de endereçamento	9
2	A Evolução do Microprocessador	11
2.1	Lei de Moore	11
2.2	Largura de palavra	11
2.3	Dependência de caminho	13
2.4	Por que a lei de Moore funciona?	15
2.5	Fabricação & Consumo de energia	16
2.6	Tecnologias de Fabricação	19
2.7	Power Wall	21
2.8	O fim da lei de Moore	22
3	Desempenho e paralelismo	22
3.1	Benchmarking	22
3.2	Lei de Amdahl	24
	Paralelizando programas	25
	Uma discussão prática	26
4	Uma nota sobre a história da computação	27

A esta altura da vida você já sabe que diferentes microprocessadores existem: o AVR (aquele do Arduino), o Intel Core 7, o AMD Ryzen 9000, o Apple M4, o ESP32, aquele lá do Raspberry Pi que varia dependendo da versão... Mas *o que significa* dizer que eles são diferentes?

Spoiler: quando o binário de dois processadores é compatível, dizemos que eles possuem a mesma ISA (*Instruction Set Architecture*), ou seja, ambos podem executar o mesmo código binário sem alterações. Isso acontece também com os processadores utilizados pelos celulares da Samsung e da Apple, embora ambos usem sistemas operacionais diferentes.

Vamos lá tentar entender estes detalhes, seguindo uma abordagem mais ou menos histórica.

1 O que é arquitetura de computadores?

Arquitetura de computadores pode ser definida como o estudo da construção de microprocessadores, incluindo especificação (ISA), implementação com circuitaria (microarquitetura) e demais considerações (periféricos, memórias, organização de multicores etc.), mas isso não é importante. Importante é definir direito uma ISA.

1.1 ISA

Aprender esta disciplina pode ser uma autópsia da história da computação; fique seguro de que todos os conceitos e técnicas vêm de algum lugar e contextos específicos. Por exemplo, a ideia de ISA apareceu no IBM System/360, para tentar consertar a bagunça da época, em que todo novo computador implicava em refazer todos os programas do zero (pois não haviam compiladores), para os novos e diferentes códigos binários.

Uma ISA (*Instruction Set Architecture*) é nominalmente a arquitetura que escolhemos para o nosso conjunto de instruções. Isso inclui todas as instruções que nosso processador deverá executar, e seus efeitos, mas não apenas isso. O nome pode ser meio confuso, pois a ISA efetivamente é uma especificação completa de todos os comportamentos observáveis do processador. Uma ISA inclui descrições de:

- os registradores disponíveis, suas funções e número de bits;
- todas as instruções, seu funcionamento (inclusive limites) e sua codificação (seus opcodes);
- organização da memória;
- comportamento do processador no momento em que é energizado (reset a frio ou a quente);
- interrupções disponíveis e resposta a elas;
- exceções (erros) e comportamento nestes casos;
- tratamento com os periféricos;

- temporização relativa ao clock e execução de instruções, se aplicável;
- etc.

Portanto podemos dizer que a ISA é a especificação do processador.

(Note apenas que temos registradores de uso geral, usados para as operações dos nossos programas (como nossas variáveis) e registradores de uso específico, como o PC (*Program Counter*, que guarda o endereço da instrução que deve ser executada), o SP (*Stack Pointer*), que indica o endereço do último elemento da pilha do sistema (é uma LIFO em uma área reservada da memória, usada para variáveis locais e endereços de retorno), o PSW (*Processor Status Word*), que armazena as flags (que indicam, dentre outras coisas, estouros possíveis em operações de ULA) e os SFR's (*Special Function Registers*), que são usados no acesso de periféricos).

1.2 Microarquitetura

Mas se a ISA é a especificação, a microarquitetura é a implementação. Por exemplo, há diversas formas de se implementar um circuito somador: em disciplinas anteriores, você deve ter visto os somadores completos e meio-somadores cascadeados, uma estratégia simples e facilmente expansível, mas bastante lenta no pior caso, com carries sendo propagados do LSB para o MSB, custando atrasos de portas lógicas em cada uma das etapas. O mais rápido seria fazer uma tabela em memória, mas para somar dois números de 8 bits a tabela teria 64 kbytes para produzir o resultado, o que consumiria bastante espaço da pastilha de silício. Modernamente, utiliza-se soma com *carry-lookahead*, que é bastante rápida no caso médio e possui um circuito razoavelmente compacto (olhe lá no livro-texto do Patterson se quiser ver o funcionamento).

Então, conforme o objetivo do processador em questão, podemos escolher estratégias mais rápidas que consomem mais energia ou estratégias mais lentas e econômicas. Um exemplo um pouco antigo mostra o contraste entre os processadores AMD Phenom versus Intel Atom; ambos os processadores, de 2008, possuem a mesma ISA, x86, e portanto podem rodar exatamente os mesmos programas, sem qualquer alteração, ou seja, o executável binário que roda no Phenom roda também no Atom. A diferença é que o Phenom foi criado para ter alto desempenho, não importando muito o consumo, portanto é um quadcore que consome 125 W, enquanto o Atom foi criado para economia, provavelmente visando a aumentar duração da bateria de tablets e netbooks, portanto tem apenas um core consumindo míseros 2 W. A diferença é só na microarquitetura.

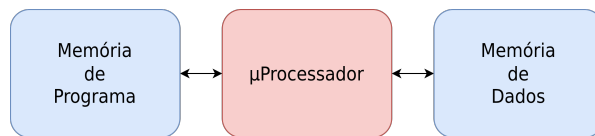


Figura 1: Arquitetura Harvard

1.3 Harvard vs. von Neumann

A primeira decisão arquitetural descrita nos livros-texto diz respeito à organização da memória: onde serão colocados os programas e seus dados? Classicamente, temos duas opções.

O modelo Harvard, nomeado assim porque é como se fazia na Universidade de Harvard nos primórdios da computação digital, separa explicitamente a área de memória de programa (classicamente uma ROM, memória só de leitura) da área da memória de dados (RAM). Cada uma possui barramento exclusivo de acesso, como visto na figura 1 (barramento ou bus é um conjunto de condutores – fios – por onde trafegam informações).

Um exemplo bem comportado disso é o antigo microcontrolador 8051, que possui instruções exclusivas para cada área: a ROM pode ser lida com MOVC (MOVE from Code memory), a RAM pode ser lida e escrita com MOVX (MOVE to/from eXternal memory). Com isso temos dois *espaços de endereçamento distintos*, o que gera a necessidade de especificar o que está sendo usado, já que “o endereço 123 da memória” pode se referir ao endereço 123 da memória de dados ou ao endereço 123 da memória de programa.¹

Já o modelo von Neumann foi utilizado no importantíssimo computador ENIAC da década de 40, cujo descendente EDSAC o John von Neumann, uma das grandes figuras pioneiras da computação, descreveu em um relatório.² Neste modelo, uma única memória contém tanto o programa quanto os dados necessários, acessados pelo mesmo barramento e mesmas instruções. O diagrama em blocos da figura 2 é meio deprimente de tão simples.³

Rigorosamente, podemos dizer que a distinção é: máquinas Harvard pos-

¹Piora: temos uma memória RAM interna e rápida para variáveis, que é acessada por instruções MOV, e portanto temos *três* espaços de endereçamento.

²A grande contribuição dele foi *documentar* as ideias, e o nome dele colocou por isso, mas os chefes de projeto era John Mauchly e John Eckert, o que demonstra os problemas de atribuição em invenções.

³Se você acha ele meio idiota, lembre-se de que ele exprime a ideia de um *programa armazenado em memória*. Anteriormente, um programa de computador era uma série de interligações de fios em um painel, como aqueles painéis telefônicos de filmes antigos com telefonistas, e codificar as instruções era, essencialmente, *alterar eletricamente o circuito do computador*, algo bastante difícil, trabalhoso, lento e sujeito a erros. As programadoras (eram todas mulheres) eram de um altíssimo nível técnico para conseguir realizar o trabalho, e os engenheiros (todos homens) nem sabiam como programar a máquina na prática. Nem preciso explicitar a moral disso.

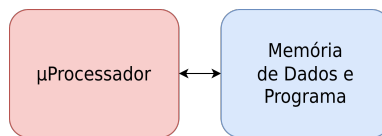


Figura 2: Arquitetura von Neumann

suem espaços de endereçamento distintos para programa e dados, máquinas von Neumann possuem um espaço de endereçamento único. A grande vantagem do modelo Harvard em relação ao von Neumann é que podemos realizar facilmente duas operações simultâneas, já que temos dois canais diferentes de transmissão de informação, e este recurso é largamente usado nos processadores atuais, por conta da exigência de alto desempenho. Raramente encontramos um processador von Neumann nos dias de hoje.

Já a vantagem do von Neumann diz respeito principalmente à otimização do uso de memória e, secundariamente, ao espaço físico utilizado (menos fios, menos CIs, menos controle necessário, menos projeto etc.). Seria estranho se fôssemos comprar dois pentes de RAM DDR para upgradear nossos PCs, sendo um para dados e outro para programas...

Haja vista o exposto, *os computadores modernos são de uma arquitetura híbrida* (podemos chamá-la de “Harvard Modificado”): possuem uma memória principal única (os pentes DDR), que como no acesso von Neumann guardam tanto programas como dados, indistintamente, mas possuem internamente possuem uma memória cache bipartida, como no Harvard, com uma área reservada para instruções dos programas e outra área para os dados acessados pelas instruções. Com isso, temos flexibilidade e desempenho juntos. (A memória cache é uma memória mais rápida colocada próxima ao processador para acelerar o desempenho e será vista oportunamente.)

1.4 Três ISA’s modernas

As duas ISA’s mais populares do mundo são, dependendo de como se contabiliza, dos processadores Arm e dos processadores x86-64 da Intel e AMD. O que iremos estudar é um processador com a ISA RISC-V, que está em ascensão constante. As três possuem grandes diferenças entre si.

A arquitetura x86 tem décadas nas costas e uma complexidade assombrosa (discussão em breve!), normalmente focada em desempenho de ponta, com a desvantagem de poder fritar ovos em cima do processador (em 2026 a Intel conseguiu evoluir neste sentido, pois estava perdendo mercado para a Arm). Mais sobre esta história à frente, na discussão de dependência de caminho.

O modelo de negócios da Intel é o mais tradicional, incluindo a concepção dos processadores, seu projeto eletrônico e de manufatura e até mesmo a fabricação – nada disso é terceirizado. A AMD é similar. Ambas com-

partilham entre si patentes que proíbem qualquer outra pessoa de fazer um processador compatível.⁴

A Arm, em contraposto aos x86, sempre foi famosa por microprocessadores extremamente eficientes com baixo consumo⁵ e recentemente (anos 20) conseguiu alcançar os x86 em desempenho nos produtos para o consumidor. Eles estão em todos os telefones celulares do mundo (são vendidos mais de um bilhão deles anualmente), o que faz muito sentido para economizar bateria.

E a forma de lidar com o mercado também é bem diferente: a Arm *não manufatura chip nenhum*, ela apenas licencia a ISA para outros fabricantes! Como é uma líder de mercado, os contratos são gigantescos, sendo ou por royalties por cada componente vendido ou por um preço anual fixo para a empresa licenciada. Em alguns modelos, a Arm já fornece todo o projeto eletrônico do núcleo e a empresa faz a manufatura, e em outros a empresa faz todo o projeto, o que possibilita algumas otimizações (este é o caso da Apple e da Samsung, o que explica eles terem processadores nitidamente melhores do que a concorrência).

Assim como no caso da Intel, você, um zezinho ou uma mariazinha, não pode projetar um circuito compatível com a ISA. Não há permissão possível para construir um x86, e não há dinheiro possível para construir um Arm. Isso fez com que pesquisadores da área ficassem substancialmente irritados por não poderem testar novas ideias na academia, fora da indústria de ponta (usualmente mais conservadora).

O nosso RISC-V é uma ISA recente e *open source*, o que significa que qualquer pessoa pode fabricar um compatível, trazendo imediatamente duas consequências: todos os trabalhos acadêmicos de Arquitetura de Computadores podem utilizá-lo e empresas não precisam pagar royalties quando necessitam de um processador qualquer dentro de seu circuito integrado. Isso criou uma onda de grandes contribuições e adoção na indústria. Além de *open source*, a ISA é *configurável* – cada fabricante pode ou não incluir determinadas características (como por exemplo, operações com floats ou não), o que é muito interessante.

No ano de 2026 já existe um Linux com interface gráfica rodando em um SBC (*Single Board Computer*, uma placa parecida com uma Raspberry Pi) com o RISC-V, apesar de meio lento. E os ESP32 mais recentes já rodam com um RISC-V neles. Os chineses, em especial, estão tendo grande adoção dele, pois lá as exigências de baixo preço são rigorosíssimas.

⁴Claro que a história é mais complicada que isso, incluindo processos judiciais cruzados entre Intel e AMD e um ou outro fabricante licenciado, sem grande proeminência.

⁵Existe uma “lenda de origem” do primeiro Arm que digamos, “consumia zero de corrente.” Tá lá um dos criadores, o Steve Furber, contando em primeira mão: vídeo do Youtube. Toda a história inicial é interessante, inclusive contada pessoalmente pela outra criadora, a Sophie Wilson.

1.5 ISA RISC versus ISA CISC: decisões de implementação

Estes conceitos são úteis apenas na medida em que nos possibilitam discutir escolhas de design para os microprocessadores, pois a classificação é um pouco ambígua quando colocada em prática.

RISC é um acrônimo que significa Reduced Instruction Set Computer. Leia bem: é um Computador que tem um Conjunto Reduzido (pequeno) de Instruções, não são as instruções que são pequenas, é a lista que tem poucas instruções. Estes processadores surgiram na década de 1980 quando alguns pesquisadores resolveram mensurar exatamente o que os processadores faziam quando estavam em funcionamento, e descobriram que algumas instruções eram usadas muitas e muitas vezes, enquanto outras apareciam muito raramente.

A ideia então foi simplificar o projeto do microprocessador, retirando instruções “exóticas” e deixando apenas o fundamental, para que isso resultasse em **ganho de desempenho**: é melhor executar rápido as coisas comuns e demorar nas incomuns. Isso facilita muito a otimização dos circuitos, em especial abrindo a possibilidade de se utilizar a técnica de pipelining, presente em praticamente todos os processadores contemporâneos.

Com isto em mente, dois projetos foram criados nos EUA e financiados pela ARPA, a agência de fomento de pesquisa dos militares (a mesma que financiou a criação da Internet) ⁶ Um era o chamado Berkeley RISC, dirigido pelo Patterson do livro-texto; o outro era o Stanford MIPS, capitaneado pelo Hennessy do livro-texto, ambos bem similares. O aposentado MIPS foi líder de mercado na área de servidores e em telecomunicações (estava em um monte de roteadores). Já o Berkeley RISC é a semente inicial para o nosso RISC-V, como você deve imaginar.

Como o Patterson criou o nome RISC, para diferenciar os processadores antigos destes novos ele criou o termo CISC: Complex Instruction Set Computers, ou seja, “o conjunto de instruções destes processadores é complicado,” meio depreciativo. Apenas numa primeira olhada, os processadores Intel atuais (como os Core 3 a 9) e o processador dos mainframes IBM System z possuem mais de 1000 instruções cada, ⁷ enquanto os RISC mais simples raramente passam de 100. Estes Intel e IBM são praticamente os únicos processadores CISC no mercado, além de alguns poucos microcontroladores de 8 bits.

O paradigma RISC, que é praticamente impossível de ser seguido à risca, preza muito pela regularidade então. Por exemplo, as instruções RISC de-

⁶Sobre o fato da inovação verdadeira sair inicialmente de fundos públicos e não de investidores privados, mesmo o Bill Gates admitiu isso a certa altura. Pesquise vídeos do Noam Chomsky se quiser ver um discurso bastante esquerdista e coerente a respeito da transferência de fundos do setor público para o setor privado.

⁷É um pesadelo, nem dá pra ter um critério claro para saber “o que é uma instrução”... Veja o blog: [How Many x86-64 Instructions Are There Anyway?](#)

vem ter todas o mesmo tamanho (32 bits no MIPS) enquanto que as do CISC variam (podem ter de 1 a 17 bytes no x86-64!), o que provoca trabalho extra de decodificação.

Talvez o mais flagrante para o programador sejam as funções típicas de uma instrução: as RISC realizam uma tarefa pequena e simples ao invés de algo complexo e composto. Veja uma instrução típica do IA-32 (assembly 32 bits do x86):

```
xor %eax,12(%ebx,%ecx,8) # sintaxe AT&T, não é Intel/AMD!
```

Isso faz uma operação ou exclusivo entre o valor do registrador %eax e o conteúdo da memória na posição dada por %ebx+%ecx*8+12; o resultado do XOR é gravado nesta mesma posição. O equivalente desta instrução em assembly RISC-V, usando s0 para %eax e assim por diante, seriam cinco instruções:

```
sll t0,s1,3    # t0 <= s1<<3, ou seja, calcula ecx*8
add t0,t0,s2    # t0 <= t0+s2, portanto t0 tem ebx+ecx*8
lw  t1,12(t0)   # t1 <= RAM[t0+12], lê da memória
xor t1,t1,s0     # faz ou exclusivo com s0 (nosso eax)
sw  t1,12(t0)   # grava resultado na RAM no mesmo lugar
```

Se quisermos piorar ainda mais a complexidade, o IA-32 possui a seguinte instrução:

```
REP MOVSB
```

Esta instrução copia um bloco inteiro da memória de uma localização para outra, podendo copiar até 64 kbytes utilizando milhares de ciclos de clock, numa instrução que só pode ser traduzida para o RISC-V utilizando-se um loop, e isso pode ser bastante problemático para se otimizar num circuito da Intel. ⁸ As instruções simples dos RISC, por outro lado, facilitam a implementação de técnicas avançadas de microarquitetura.

Além disso, os RISC acessam a memória de forma simples, usando exclusivamente uma instrução tipo LOAD para ler um dado e uma instrução STORE para escrever um dado, sem realizar nenhuma outra operação. Os CISC típicos, por contraste, podem realizar múltiplos acessos de memória com uma única instrução, além de cálculos com o dado, como no XOR exemplificado anteriormente.

⁸Mas será que o REP MOVSB não fica mais rápido, já que o processador sabe que é um loop? “The ‘slowness’ is only due to the lack of attention, by the processor engineers, to the REP; MOVS technique of moving data.”, critica um sujeito qualquer em um fórum da Intel. Outro responde “On most of the newer CPU’s, a simple loop moves memory faster than MOVS.” e o engenheiro da Intel comenta embaixo: “As [the other answer] says, the simple loop, especially unrolled, is faster. Believe me, this is the sort of thing we pay VERY close attention to.”

Finalmente, as ISA RISC são ortogonais, o que significa dizer que uma instrução pode manipular qualquer um dos registradores disponíveis.⁹ Os CISC, tipicamente, possuem instruções que limitam o acesso a registradores específicos. Veja uma soma no microcontrolador 8051:

`add A,R3`

Esta instrução soma o registrador R3 ao registrador A e o resultado é gravado em A. Este registrador A é especial e é denominado acumulador nos CISC; todas as operações lógico aritméticas obrigatoriamente utilizam o acumulador como uma das fontes e como o destino da operação, ao contrário do RISC-V, que pode somar quaisquer registradores e gravar o resultado em qualquer registrador.

Um resumo da discussão pode ser visto na tabela 1. Dá pra argumentar mais algumas características se desejado (CISC's tipicamente possuem tipos de dados mais complexos, por exemplo).

Característica	RISC	CISC
Quantidade de instr.	Pequena	Grande
Tipo de instruções	Simples	Complexas
Formatos binários	Regulares	Irregulares
Acesso à memória	Modelo Load/Store	Acessos complexos
Codificação e opcodes	Regulares	Irregulares
Ortogonalidade	Instr. ortogonais	Instr. não-ortogonais (acumulador etc)

Tabela 1: RISC vs CISC

Uma vez que RISC e CISC são apenas modelos idealizados, é discutível o quanto cada processador se aproxima do ideal. Em especial, os processadores Arm se dizem RISC mas possuem várias características bastante CISC, fugindo do modelo LOAD/STORE e executando mais de uma operação por instrução, como na instrução de transferência de dados entre vários registradores e bloco de memória de uma vez só ou então no autoincremento de ponteiros (ou seja, um registrador usado no acesso à memória pode ter seu valor alterado logo antes ou logo depois da instrução, se o programador desejar). E há alternativas mais ou menos obscuras a estes modelos, como o VLIW, explicado posteriormente.

1.6 Modos de endereçamento

Dada a importância dos acessos à memória, especialmente quando tínhamos poucos registradores, os processadores antigos desenvolveram esquemas

⁹O termo “ortogonalidade” é usado no sentido em que um operando não depende da instrução – o operando é “ortogonal” a ela, como dois vetores ortogonais em álgebra linear.

sofisticados de operações com o objetivo de acelerar os programas (uma instrução complexa fazia mais coisas do que uma simples e executava mais rápido do que duas simples na era CISC). Então formas elaboradas foram desenvolvidas.

Os modos de endereçamento são as formas presentes no microprocessador para acessar dados para as operações; um modo especifica de onde vem a informação. Por aqui, ficamos apenas com o que está presente no RISC-V, suficiente para nossa discussão, a seguir.

- Endereçamento **imediato**: o dado está presente como uma constante dentro da própria instrução. No RISC-V, é algo como `addi s2,s5,76` $\Rightarrow$ a constante 76 está guardada dentro dos 32 bits do código de máquina da instrução.
- Endereçamento via **registrador**: o dado está presente em um dos registradores internos, como em `add t5,s2,t0` $\Rightarrow$ os três dados relevantes se referem a registradores.
- Endereçamento **direto**: o dado está presente em memória e nós damos na instrução o endereço onde acessá-lo. A pseudoinstrução do RARS `lw s1,12004`, por exemplo, lê o dado do endereço 12004, que é uma constante imediata.
- Endereçamento **indireto**: é o exato equivalente do uso de ponteiros em linguagem C – os ponteiros são, de fato, traduzidos para instruções assim. Neste caso, um registrador contém o valor do endereço de memória onde o dado de interesse se encontra. Uma pseudoinstrução como `lw t3,(s6)` ilustra o ponto: `s6` contém um número, e este número é o endereço de memória de onde vamos ler o dado, ou seja, `s6` é um ponteiro.
- Endereçamento **indireto indexado**: similar à anterior, mas podemos indicar um “delta” de uma quantidade de endereços a ser somada/saltada. No formato nativo do RISC-V, um `sw s0,2052(t3)` vai armazenar o dado de `s0` no endereço obtido pela soma de `t3` com a constante 2052 (a constante é imediata, neste caso – está dentro da instrução em si).

Há outros modos mais complexos, mas a maioria dos processadores apresenta no mínimo os cinco modos listados acima. Algumas variantes simples podem ser interessantes, como endereçamento indireto indexado por registrador (no z80, por exemplo, temos `LD (HL+IX),A` que armazena `A` no endereço dado pela soma de dois registradores, o `HL` e o `IX`, criando possibilidades interessantes de uso) ou dados implícitos pela instrução (o topo da pilha). Endereçamentos indiretos com autoincremento e autodecremento

dos ponteiros são populares, pois ajudam a construir pilhas, que são fundamentais para os códigos gerados por compiladores.

É preciso mencionar que o modo indexado é muito útil para orientação a objetos, pelo simples fato de que os atributos de uma dada classe (ou seja, as variáveis membro de um objeto) sempre estarão na mesma ordem na memória. Assim, se tivermos um ponteiro `ptr` para um objeto `obj` que possui os campos inteiros `x`, `y` e `z`, sabemos que `x` sempre estará no endereço apontado por `ptr`, e, considerando que cada `int` ocupa 4 endereços de memória, `y` estará 4 endereços para frente e `z` estará 8 endereços deslocado do início dos dados do objeto, de forma que `obj.z` é, na verdade, equivalente a fazermos `*(ptr+8)`.¹⁰ Desta forma, um acesso a `z` sempre será tratado como `ptr->z`, o que é traduzido para o assembly para uma instrução similar a `lw s0,8(s1)` (ou `sw` para a escrita), com `s1==ptr`.

Na Intel, os vários modelos de organização de memória (em especial o obsoleto modo segmentado) dão origem a modos de endereçamento bastante desagradáveis de se aprender e fixar.

2 A Evolução do Microprocessador

Nada no universo é criado no vácuo, e para entendermos o presente da tecnologia (e especularmos loucamente sobre seu futuro), é necessário ter alguma ideia de como as coisas aconteceram.

2.1 Lei de Moore

Nenhuma discussão sobre a evolução dos computadores está completa sem a Lei de Moore.¹¹ Ela possui vários enunciados semelhantes, mas ficaremos com um dos mais comuns: “*A capacidade máxima de colocarmos transistores em um circuito integrado dobra a cada dois anos.*”

Isso parece implicar num desempenho que aumenta exponencialmente e um custo por transistor que decresce exponencialmente, o que parece bastante anti-intuitivo para qualquer engenheiro. Surpreendentemente, a Lei de Moore continua sendo obedecida, de uma maneira ou de outra, veja lá na figura 3.

2.2 Largura de palavra

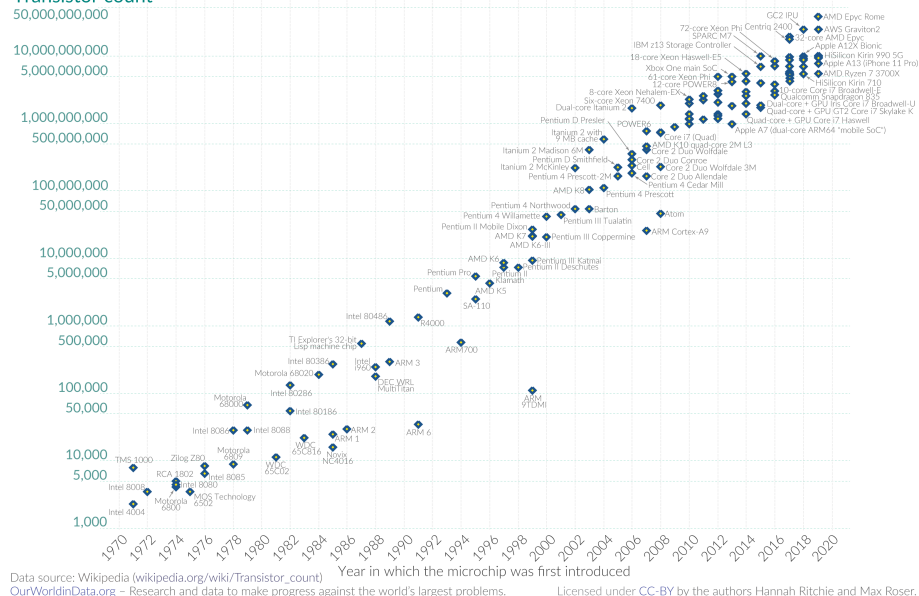
O início da era dos microprocessadores (um processador colocado em uma única pastilha dentro de um Circuito Integrado) se dá com o Intel 4004: um

¹⁰Por simplicidade, estou usando um ponteiro `char*` ou um `void*`, que permite somar endereços diretamente. Se `ptr` for um ponteiro para `obj`, aí entramos em aritmética de ponteiros, que foge do nosso assunto. Assuma que o texto omitiu algumas conversões de tipo, com minhas desculpas.

¹¹Gordon Moore foi um dos fundadores da Intel, depois de trabalhar com o William Shockley, que famosamente era um péssimo chefe e péssimo ser humano.

Our World
in Data

50,000,000,000



processador com largura de palavra de 4 bits criado para ser utilizado em calculadoras, que à época utilizavam código BCD em sua maioria.

Já em seguida, o 8008 aumentou para 8 bits a palavra, e o 8086 aumentou para 16. Não faz tanto tempo atrás que os processadores da Intel adotaram 64 bits. Quais as vantagens de se ter uma largura maior para a palavra?

Fora dos PCs, estes Arm de 32 bits dominam o mercado, especialmente

12

na telefonia celular e em sistemas embarcados. A expressão de mais números (de 0 a $2^{32} - 1$) não só traz mais precisão e contas mais complexas, mas possibilita endereçarmos mais memória. Se tivermos um ponteiro de 32 bits, podemos endereçar 4 Gbytes de memória, em oposição a 64 kbytes se usarmos apenas 16 bits.¹³

As arquiteturas de 64 bits, como as dos Intel e AMD de nossos laptops mais recentes e de alguns Arm, dobram o poder expressivo dos números. Ponteiros agora gastam 64 bits, portanto consomem 8 bytes para serem guardados numa variável (o que não é importante num PC, mas este gasto extra pode fazer diferença num sistema embarcado pequeno), e deste modo podem endereçar teoricamente até 16 exabytes ($16 * 10^{18}$).

Como curiosidade, as arquiteturas antigas possuíam números diferentes de bits, não necessariamente múltiplos de 8 (veja na Wikipédia – em inglês – os DEC PDP com 12 ou 18 bits ou <https://en.wikipedia.org/wiki/36-bit>). Com o sucesso do IBM System/360 na década de 60, considerado um dos sistemas seminais da computação, com mais influência na história, outros fabricantes adotaram o particionamento da informação em unidades de 8 bits, ao ponto que é raríssimo encontrar memórias que fogem do padrão de ter um byte por endereço.

2.3 Dependência de caminho

Uma das curiosidades que pode ser observada facilmente no gráfico da Lei de Moore é a sequência de processadores criados pela Intel. Um dado um pouco assustador é que o código binário criado para rodar num processador 8088 de 1979 pode, em teoria, rodar sem alterações no Core 9 mais recente do mercado. Note que não é o programa em assembly, são os mesmos opcodes, que os processadores são obrigados a rodar sem alterações por uma questão de retrocompatibilidade, pois nenhum usuário gostaria de trocar de computador e ter de comprar novamente todos os programas nativos que possuía por conta de uma mudança arquitetural.¹⁴

Se considerarmos que o 8080 tinha um assembly parcialmente compatível com o do 8088, e que foi fortemente baseado em ideias vindas do 8008, que por sua vez foi uma expansão do 4004, podemos dizer que temos o DNA do primeiro microprocessador dentro dos nossos laptops...

Em respeito à Intel, eles tentaram por três vezes abandonar a arquitetura x86, que obviamente é inchada e cheia de características obsoletas que complicam desnecessariamente o circuito. Na primeira, criaram o ambicioso iAPX-432, um fracasso tão retumbante que foi expurgado da história

¹³Aqui estou usando uma convenção simples: 1 kbyte = 1024 bytes, 1 Mbyte = 1024*1024 bytes e assim por diante. Existem convenções que tentam resolver esta ambiguidade de notação, mas nenhuma delas teve adoção ampla.

¹⁴Hoje, com outros modelos de negócios, como assinaturas de software e apps na nuvem, isso não é mais tão importante quanto já foi.

oficial da Intel; com um assembly orientado a objetos e outras inovações interessantes, mostrou-se exageradamente lento.¹⁵ As outras duas foram melhores: os i960 de arquitetura RISC, que foram populares em sistemas embarcados (foram usados nas impressoras laser da HP por muito tempo, por exemplo) e o Itanium, com suas instruções VLIW nem RISC nem CISC (veremos mais à frente), que teve uma fatia não negligenciável do mercado de servidores, embora isso provavelmente se deva a contratos comerciais não muito sábios.

Se você está achando estranho o processador líder no segmento de computadores pessoais ter uma arquitetura CISC defasada, que dificulta otimizações e torna os circuitos mais lentos, tem razão. E dizer que o mercado e a retrocompatibilidade forçaram a mão da Intel é correto, mas não conta toda a história. (Aliás, a AMD também tem parte da responsabilidade nisso tudo, sendo efetivamente cocriadora dos CIs atuais).

O nome deste fenômeno é “dependência de caminho”, e é estudado por economistas, que o definem aproximadamente como características de um produto sendo submetidas a denominadas externalidades do processo e não apenas à razão, caracterizando um sistema estocástico (ou seja, com memória). E o exemplo padrão usado é o formato dos nossos teclados: o QWERTY.

Na segunda metade do século XIX, máquinas de escrever de vários fabricantes competiam no mercado americano, cada uma com seu layout de teclas diferente e acionamento particular. Ocorre que a mais popular se provou a Remington, com o design que conhecemos hoje, um retrabalho da disposição original criada por um certo inventor (a ideia popular de que ele foi criado para se dificultar a digitação, não é suportada por documentos, embora seja verdade que os martelos que imprimiam os tipos na fita se chocam uns com os outros no caso de digitação muito veloz). Claro que deixar as letras E e A, que estão entre as três mais frequentes no inglês, a cargo dos dois dedos menos hábeis de uma pessoa destra, não parece ser muito eficiente para olhos modernos.

Como na época havia mais gente que dominava o teclado QWERTY, mais cursos de datilografia ofertados para o padrão, mais máquinas disponíveis, novos usuários eram atraídos, já que retreinamento custa dinheiro e o domínio de padrões menos usados impacta na empregabilidade do funcionário e na dificuldade de achar mão de obra qualificada para o patrão.

Portanto, até hoje utilizamos um esquema meio arbitrário de teclas, criado há cerca de 150 anos, longe de ser o melhor possível, por pura e simples dependência de caminho: o padrão é definido pelo caminho histórico que trouxe o produto até aqui e não por características otimizadoras e sensatas, e não pode ser alterado sem grande custo.

O mesmo ocorre com os processadores Intel. O impacto é tão grande que,

¹⁵Outros processadores exóticos existem, como os baseados em pilha e os assíncronos.

a certa altura, os engenheiros se viram obrigados a criar um outro código de máquina, como um assembly interno, RISC e invisível ao programador, que pode usar técnicas modernas de otimização, enquanto mantém o assembly externo CISC, retrocompatível, sendo traduzido por circuitos extras para o interno.

Um exemplo mais divertido, muito ilustrativo porém não absolutamente factual, é o da envergadura dos ônibus espaciais americanos, em especial a envergadura dos tanques, sendo determinada pelo diâmetro dos túneis de trem, que dependeriam do transporte no império romano antigo.

2.4 Por que a lei de Moore funciona?

Pode ser surpreendente que uma lei, aparentemente de cunho prático, seja exponencial e ainda assim corresponda tão bem à realidade por mais de 50 anos. No mínimo, parece um otimismo exagerado.

A explicação, no entanto, é simples. O desenvolvimento de um novo microprocessador é bastante longo, levando entre 4 e 5 anos para uma nova geração de Intel Core, por exemplo, o que pode incluir mudanças na tecnologia de fabricação. Estimativas colocam o custo de uma nova fábrica com tecnologia de ponta, necessária quando muda o processo e há redução de tamanho dos transistores, em dezenas de bilhões de dólares, e ainda por cima são necessárias algumas fábricas, não uma só.

Portanto, quando o lançamento de um novo processador é planejado inicialmente, a empresa deve tentar adivinhar qual será o estado do mercado cinco anos à frente, algo impossível de ser feito. O resultado é que toda a indústria sempre se pautou na Lei de Moore para ter uma noção (os grandes *players* de setores tecnológicos usualmente produzem o chamado Roadmap da indústria, com as previsões para os próximos anos). Portanto tanto a Intel como seus concorrentes possuem previsões similares e metas similares, o que não impede que se cometam erros: uma das razões para a AMD perder mercado na década de 10 foi chutar muito baixo (diga-se de passagem, as inovações da Intel também não foram fáceis de competir). Em torno do ano 2000, foram as inovações da AMD que deixaram a Intel para trás.

Portanto, neste sentido, a Lei de Moore é uma “profecia autocumprida.” Apesar disso, quanto mais perto chegamos dos limites físicos dos semicondutores, mais ameaçada está essa evolução. A Intel recentemente decidiu atrasar o lançamento de uma nova tecnologia de fabricação por conta dos problemas técnicos que a miniaturização está trazendo.

Se você ainda acha que existe alguma lei econômica ou tecnológica que dita este processo, dê uma olhada no gráfico da figura 4, retirado do artigo original de 1965. Peralá, 1965?! Pois é, nem existia microprocessador ainda... Perceba que o Moore tinha *quatro* pontos de amostragem apenas, portanto nunca pensou que era uma “lei.” Deveria se chamar, mais corretamente, “A Observação de Moore.” Interessante perceber que, já nesta época,

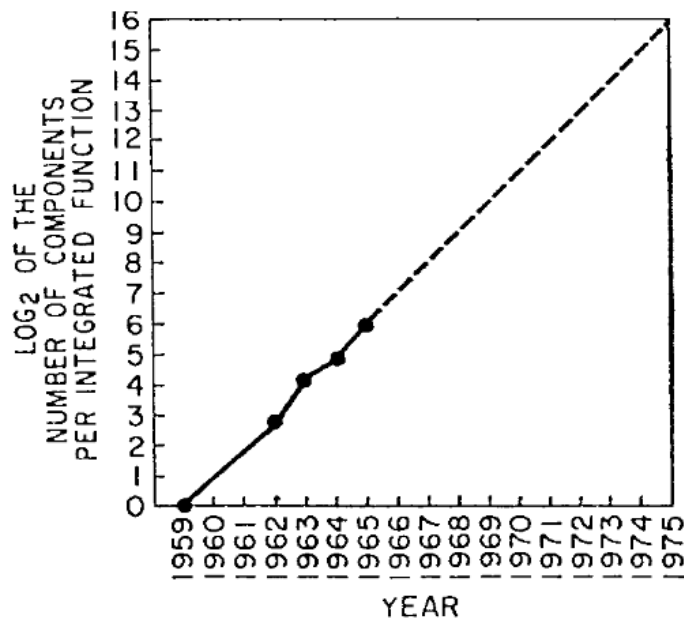


Fig. 3.

VII. HEAT PROBLEM

Will it be possible to remove the heat generated by tens of thousands of components in a single silicon chip?

Figura 4: Gráfico da lei de Moore do artigo original

ele menciona o problema de aquecimento dos chips, que vamos ver agora.

E é engraçado perceber o número de componentes que era possível colocar em um CI na época (50), e pensar que assim mesmo isso influencia a forma como manejamos este desenvolvimento tecnológico.

2.5 Fabricação & Consumo de energia

As forças irmãs do aumento de desempenho e da diminuição do consumo formam o principal impulso para o desenvolvimento de novas tecnologias de microprocessadores. Economizar energia é um desejo óbvio, pois queremos que as baterias de celulares e laptops durem mais, mas há também grandes consequências quando se pensa em maior escala, especialmente em grandes *data centers* como os utilizados para armazenar dados e processar serviços, como os do Google ou Amazon.

Já o aumento de desempenho parece ser simples de entender: queremos

que os celulares e PCs executem as tarefas mais rapidamente e que, com mais velocidade, novas tarefas, mais pesadas, se tornem possíveis. Mas temos a “lei de Wirth” ¹⁶ talvez um pouco irônica, dizendo que a complexidade do software mais do que duplica a cada dois anos, tornando a lei de Moore ineficaz.

De qualquer maneira, fica ainda a questão: como aumentar o desempenho e diminuir o consumo?

Infelizmente, estas duas variáveis formam um *trade-off*, ou seja, um sistema de concessões mútuas: se uma melhora, a outra deve piorar e vice-versa. A razão para isso é que consumo maior e desempenho melhor são praticamente intercambiáveis, e a principal razão para isso é a tecnologia de fabricação de circuitos integrados.

Atualmente são utilizadas variantes da tecnologia CMOS (Complementary Metal Oxide Silicon) para construir os transistores que formam a base dos circuitos digitais: portas lógicas e células de memória. O consumo destes circuitos é dado por:

$$P_{CMOS} = P_{est} + P_{din}$$

onde P_{est} é a potência dissipada estaticamente, ou seja, quando o circuito está alimentado mas nenhuma alteração ocorre nele, e P_{din} é a potência dinâmica, medida quando o circuito está em operação, mudando valores. Desculpem a equação simplória, mas...

Uma estimativa de 2016 dava a potência estática como consumindo cerca de 40% da potência nominal de um processador, algo nada insignificante. Este gasto ocorre com o processador ligado mesmo que ele esteja ocioso, pois é essencialmente causada pelas correntes de fuga nos transistores, com portadores atravessando regiões que deveriam estar isoladas, com corrente igual a zero. Quanto menores os transistores, menor é a distância que o elétron precisa percorrer para “escapar” e portanto maiores essas correntes indesejadas, ou seja, hoje em dia a situação é ainda pior.

Já a potência dinâmica é dada pela equação:

$$P_{din} = V^2 * f * C$$

sendo V a tensão de operação, f a frequência de chaveamento e C a capacitância de junção. Várias observações importantes relativas a esta equação são numeradas a seguir.

1) O consumo é proporcional à frequência

Isto se deve ao fato de que, num circuito CMOS, temos um FET de pull-up entre saída e alimentação e um FET de pull-down entre saída e massa, como se vê na figura 5. Um FET é um transistor de efeito de campo e,

¹⁶Do Niklaus Wirth, o criador da linguagem Pascal; há várias outras semelhantes.

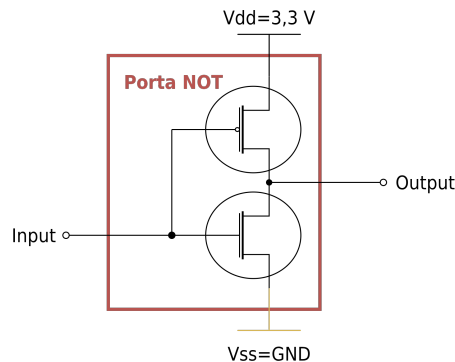


Figura 5: Implementação de uma inversora com transistores CMOS

simplificadamente, funciona como uma chave liga-desliga: se jogamos um nível 1 nele (no gate) ele curto-circuita os outros dois terminais (saturação) que formam o chamado canal, caso contrário eles estão em aberto (corte). Na figura, o pino input está ligado no transistor inferior, então com nível 1 nele, V_{SS} será conectado à saída; com nível 0 nele, V_{SS} fica desconectado dela. O FET superior da figura é invertido (tem uma bolinha pequena na entrada, olhe bem), então irá curto-circuitar V_{DD} ao pino output quando a entrada estiver em 0 e isolar V_{DD} quando em 1. O resultado é a inversão do estado lógico, ou seja, $output = \overline{input}$.

Como as impedâncias envolvidas são ou altíssimas ou baixíssimas, a corrente é praticamente zero quando o circuito está numa situação estável, consumindo apenas a potência estática devida às correntes de fuga. Porém, quando há um chaveamento, um dos FETs muda de ligado para desligado e o outro faz o contrário, simultaneamente; em determinado instante, ambos estão conduzindo parcialmente, com resistência de canal muito baixa, o que provoca uma corrente significativa por um curto período de tempo. Portanto, quanto mais transições a porta lógica fizer, maior a potência dissipada.

É interessante notar que para obtermos maior desempenho de um microprocessador, basta aumentar a sua frequência de operação (*caveat emptor*: observe a Lei de Amdahl mais à frente), pois se dobrarmos o clock, teremos potencialmente o dobro de instruções sendo executadas num mesmo período de tempo. Assim, se o projeto do processador permitir aumentar a frequência de operação sem destruí-lo por excesso de temperatura devido ao efeito Joule, podemos aumentar seu desempenho facilmente.

2) Já que o a tensão V é ao quadrado, ela deve ser o primeiro foco

Isto explica como passamos dos 5V de alimentação nos anos 80 para cerca de 1,1V nos processadores modernos de alto desempenho. Abaixo deste valor, as tensões ficam difíceis de gerenciar pois as faixas de níveis lógicos ficam muito pequenas e os circuitos se tornam suscetíveis demais a ruídos.

3) A capacitância C diminui de acordo com o tamanho do transistor

Todo novo processo de fabricação de circuitos integrados diminui o tamanho dos componentes, através de melhorias no processo de máscaras e gravação em silício. Quanto mais diminui o tamanho dos transistores FET, menor é a largura do canal e tamanho do gate; portanto, a distância entre os contatos diminui, ou seja, temos menos material dielétrico, terminais menores e portanto menor capacitância de junção.

Assim, uma das vantagens da miniaturização progressiva é a diminuição proporcional da potência dinâmica. Portanto, nos interessa diminuir o máximo possível os transistores, embora isso aumente o consumo estático devido às correntes de fuga.

2.6 Tecnologias de Fabricação

As tecnologias de produção de circuitos integrados são usualmente expressas em nanômetros e, via de regra, o estado da arte na indústria é representado pelos processos empregados nos microprocessadores comerciais. Em 2024, as tecnologias de ponta eram das empresas TSMC, Global Foundries. Intel e Samsung e são descritas como sendo de “3 nm,” mas isso é mais uma questão de marketing ¹⁷ do que uma medida de tamanho de algo no transistor. ¹⁸ As quatro empresas utilizam maquinário produzido exclusivamente pela holandesa ASML, que por sinal é proibida de exportar para a China por pressão comercial dos EUA.

Portanto, para os microprocessadores, o “número de nanômetros” da tecnologia é como uma marca de fantasia. As dimensões de interesse dos transistores estão em algumas dezenas de nanômetros (em 2024, a distância

¹⁷O que mais interessa para os processadores é a quantidade de transistores por mm^2 que podemos atingir; neste quesito, os concorrentes estão aproximadamente equiparados. Outros parâmetros também são relevantes, em especial relativos a resfriamento.

¹⁸No caso de memórias RAM, o número é definido como o “half-pitch” (meia-distância entre elementos idênticos numa matriz) para uma célula de memória. Uma célula é, essencialmente, um transistor e um capacitor; se considerarmos ainda o espaçamento entre as células, uma distância de 14 nm é aproximadamente a largura de um bit, ou perto da largura de um transistor. Isto é complicado atualmente pelo fato de que algumas memórias têm os bits empilhados verticalmente, e pelo fato de que a fabricação de memórias não se beneficia muito da miniaturização que ocorre nos microprocessadores.

mínima entre gates de dois transistores era de 48 nm e entre conexões é de 24 nm, por exemplo), apesar dos “3 nm” do nome.

Se considerarmos a exigência prática da lei de Moore, isso parece implicar numa diminuição exponencial das dimensões envolvidas, o que se torna extremamente problemático: dentro de uma estrutura cristalina semicondutora, a distância entre o centro de dois átomos adjacentes de silício é cerca de 0,22 nm; a largura do canal (*fin width*) dos FinFETs da já superada tecnologia Intel 10 nm é cerca de 7 nm, o que podemos supor que resulta em cerca de 32 átomos de largura por onde passa a corrente do sinal. Isso dá uma margem minúscula de erro, e é uma das causas do baixíssimo aproveitamento (*yield*) das pastilhas semicondutoras fabricadas em um wafer de silício.

Ao compreender o processo básico de fabricação, essa impressão piora: até hoje usamos variantes da fotolitografia utilizada nos primeiros CIs nos anos 70... Isso consiste em termos um substrato isolante na camada inferior, coberto por uma camada de material semicondutor ativo. Utiliza-se então uma máscara com o negativo do desenho do circuito, como as máscaras de PCB (printed circuit boards) ou de silkscreen para camisetas (serigrafia), e processos químicos para criar as junções semicondutoras. Todos os componentes ativos estão num mesmo plano, como numa folha de papel: a distribuição deles, portanto, é feita em apenas duas dimensões.

Por cima destas camadas, que formam transistores, resistores e capacitores, são colocadas outras camadas bidimensionais com contatos entre si para fazer o roteamento das ligações, bem como os planos de alimentação e terra. Perceba que uma consequência deste arranjo é que a dissipação de calor vai seguir o módulo do circuito que está sendo usado, e a distribuição destes módulos é bidimensional, obviamente. Para uma introdução à fabricação de microprocessadores, há um vídeo excelente em <https://youtu.be/dX9CGRZwD-w>.

Estamos muito lentamente caminhando para 3D, com diversas variações chamadas de “dois D e meio.” A Intel usa, já desde a tecnologia de 22 nm, “transistores 3D”, que na verdade continuam todos no mesmo plano, apenas possuem o gate empilhado de alguma forma (FinFETs e gate-all-around); fabricantes de FPGAs em 2010 começaram a fazer “empilhamento de pastilhas” – módulos com funções diferentes colocados em alturas variadas dentro do encapsulamento e interconectados por vias correndo as três dimensões; e memórias de alto desempenho e alta capacidade colocam pastilhas idênticas em uma pilha, na mesma posição e com conexões enormes entre elas para facilitar alinhamento e condução térmica.¹⁹ CIs verdadeiramente em 3D, no entanto, com um circuito sendo decomposto em várias alturas diferentes de transistores no mesmo encapsulamento, continuam sendo apenas tópico

¹⁹ Até o ano de 2024, chegaram a ser usadas 200 camadas em uma memória em casos extremos.

de pesquisa em laboratórios, com múltiplos desafios tecnológicos.

2.7 Power Wall

Retornando ao gráfico da Lei de Moore, uma coisa é notável no início dos anos 2000: o surgimento de chips multicore. Mas há um porém, pois apesar do número total de transistores continuar seguindo a previsão, a quantidade em um núcleo único dentro de um microprocessador não continuou a escalada exponencial. Então aparecem processadores comerciais inicialmente com dois, depois quatro, oito núcleos dentro de um mesmo encapsulamento, com contagens de bilhões de transistores em uma área usualmente entre 100 a 400 mm².

Mas por que razão mudar para multicore? O desempenho não poderia continuar sendo aumentado utilizando-se um processador único como anteriormente?

A resposta é não. Neste ponto da história, a indústria alcançou o limite prático de resfriamento para processadores comerciais, utilizando dissipadores metálicos e ventiladores potentes: se não for usada alguma técnica cara ou impraticável (nitrogênio líquido, digamos), processadores trabalhando em clock elevado vão aquecer por efeito Joule e a temperatura excessiva leva à ruptura mecânica da pastilha de silício. Portanto as frequências de clock usadas ficaram muito abaixo do que seria possível ou desejável.²⁰

Este momento de conversão para multicore é conhecido como Power Wall.

A partir de então, a evolução continuou, tanto da circuitaria para cada núcleo do processador quanto das frequências de clock, mas num passo bem mais modesto, fazendo com que a velocidade final do processador aumente pouco, em comparação com os aumentos escandalosos obtidos na década de 90. Nos dias de hoje, o usual é que cada nova geração de processadores melhore apenas um pouco em cada área de interesse: clock, microarquitetura, consumo, taxa de transferência de dados, velocidade das memórias. Isso gera um aumento perceptível de velocidade de processamento, mas não um aumento brutal.

Tipicamente, o número de núcleos (microprocessadores independentes dentro de um mesmo encapsulamento) tende a aumentar no mercado destinado a consumidores caseiros: começamos com os dual core em torno de 2006, vinte anos depois os octacores ficaram comuns. No mercado de servidores, IA e jogos de alto desempenho, várias dezenas de núcleos em um

²⁰Tecnicamente, isso é relacionado ao limite do chamado escalonamento de Dennard, que nos diz que, com a redução do transistor, a *densidade de potência* do componente (ou seja, quantos watts por mm² são dissipados) continua a mesma e portanto o resfriamento não precisa ser melhorado. No tamanho do transistor do início do século, esta regra prática (ou “lei”) parou de funcionar e portanto os ganhos de frequência sofreram uma parada abrupta.

único encapsulamento não surpreendem mais.

2.8 O fim da lei de Moore

Uma aparente maioria diz, sem meias palavras, que a lei de Moore já acabou faz tempo, mas não há um consenso verdadeiro sobre isso, especialmente porque a *ideia* por trás da lei era tentar entender a relação dos avanços em microeletrônica com diminuição de preços e aumento de desempenho. Se tomarmos uma das versões ao pé da letra, aquela que supostamente diz que “o número de transistores em um núcleo de microprocessador dobra a cada dois anos” (é uma paráfrase forte, mas capta o espírito), claramente isso parou de acontecer desde o surgimento dos multicores no mercado para consumidores.

No entanto, o gráfico nos mostra que realmente continuamos uma taxa exponencial de aumento no número de componentes em um mesmo encapsulamento,²¹ embora esta taxa tenha diminuído. Esta é a visão oficial da Intel.

Uma visão menos reducionista da situação é que ainda temos aumento contínuo de desempenho dos nossos sistemas computacionais, com ou sem lei de Moore, e a miniaturização e empilhamento de transistores continuam rendendo, e todo novo passo de melhoria traz incógnitas e problemas sem solução óbvia.

Avanços “disruptivos” nunca podem ser previstos e, nos anos 20, costumam ser relacionados a nichos da computação e inaplicáveis a computação de uso geral. Exemplos de tecnologias emergentes, que podem ou não dar resultados, são computação fotônica (em especial para IA) e computação quântica (para simulação de processos físicos).

3 Desempenho e paralelismo

Toda a nossa questão no projeto de microprocessadores se resume ao simples fato de que qualquer usuário quer computadores mais rápidos. Há detalhes não tão claros, no entanto.

3.1 Benchmarking

Discutindo a relação da frequência de clock do microprocessador com o consumo, mencionamos de passagem que quanto maior o clock, mais instruções por segundo podemos executar. Podemos medir este valor em MIPS (Millions of Instructions per Second), mas algumas observações são pertinentes sobre esta medição.

²¹Tecnicalidade: os processadores comerciais atuais são efetivamente uma placa de circuito impresso (PCB) na qual várias pastilhas de silício são interconectadas, o que deixa um pouco ambígua a ideia de circuito integrado.

Em primeiro lugar, os conjuntos de instruções de processadores diferentes podem ser muito diferentes: comparar um RISC-V com um Intel Core pode não fazer muito sentido, pois as instruções x86-64 são notavelmente mais complexas do que as RISC. Em segundo lugar, estes números são tipicamente valores de pico fornecidos pelos fabricantes, que obviamente escolhem a instrução mais rápida para a medição (sempre o NOP, “no operation”, disponível em todo microprocessador): portanto, o número em si pode não ser muito representativo.

A comparação de desempenho entre dois sistemas é conhecida como *benchmarking*, nome derivado das hastes à beira de rios utilizadas para indicar qual o nível das águas atingido, em metros. O uso de medidas relativas (comparando especificamente um sistema X com outro sistema Y) parece ser indicado, portanto, já que as medidas absolutas, como o MIPS, são questionáveis.

Uma alternativa é medir a quantidade de instruções usando programas específicos. O conjunto de testes conhecido por SPEC, por exemplo, usa programas reais que exigem bastante do processador, como compiladores, compressores de vídeo e programas para cálculos pesados de física e química. Um dos subtestes SPEC avalia a capacidade de um sistema completo de servidores de responder requisições externas, medindo transações por segundo. Já os testes CoreMark permitem a customização das tarefas a serem testadas, escolhidas dentre um rol de atividades comuns para sistemas embarcados (por exemplo, implementação de um pilha TCP/IP ou descompressão de vídeo h.264).

Apesar disso, duas medidas absolutas ainda são bastante utilizadas. FLOPS (Floating Point Operations per Second), usualmente medido em TFLOPS, que é bastante adequadas para aplicações científicas de “number crunching”, essencialmente contas e mais contas com algoritmos numéricos (digamos, aproximação numérica de equações diferenciais para cálculo de reatores nucleares e coisas similares a isso). Dado este domínio, essa medida também é interessante na área de computação de alto desempenho (HPC - *High Performance Computing*), desde que seja uma medida em casos médios ao invés de pico. Normalmente é utilizada uma biblioteca ou pacote chamada Linpack, com rotinas de álgebra linear e matrizes que costumam ser fundamentais para estas aplicações.²²

A outra medida ainda utilizada, dentro do segmento de sistemas embarcados, é o Dhrystone MIPS, ou DMIPS. Ela indica a quantidade de instruções executadas por segundo, em milhões, para um teste específico: um programa dito sintético, consistindo de uma sequência de tarefas consideradas representativas de um hipotético “caso médio” para uma tarefa genérica, consistindo de contas, cópias de strings, loops e outras, chamado

²²Para informações sempre atualizadas sobre os maiores supercomputadores, cheque as listas de <http://top500.org>.

de benchmark Dhrystone. Infelizmente este caso médio não vai ser exatamente balanceado para todas as aplicações: um banco de dados usa muito o sistema de arquivos, uma simulação em física usa quase exclusivamente cálculos em ponto flutuante, algoritmos de busca farão muitas condicionais. O número final, portanto, serve apenas para dar uma ideia da capacidade do sistema em questão.

Tipicamente, as medidas são fornecidas em DMIPS @ Hz, como: “400 DMIPS @ 1 GHz”. Isso ocorre porque o processador pode ser colocado para funcionar em uma gama de frequências, e por questões de economia de energia, talvez seja usado numa frequência muito inferior à frequência máxima. Os manuais, obviamente, trazem os números obtidos com o processador no talo das suas capacidades.

Cabe mencionar que há outros benchmarkings interessantes, incluindo alguns que medem indiretamente o impacto ambiental de Data Centers grandes, como os do Google, usualmente construídos ao lado de rios para poderem utilizar a água para resfriamento das instalações e equipamentos (como nota ecológica, cabe ressaltar que a água aquecida pelo centro é transferida para piscinas antes de ser devolvida ao rio, para evitar impacto ambiental: a elevação de temperatura poderia ser prejudicial à vida local). Outros medem a eficiência energética de processadores, algo que é importante tanto para as baterias dos nossos celulares quanto na supercomputação.

3.2 Lei de Amdahl

Se o desempenho de um sistema depende de tantas variáveis, como velocidades do processador, memória, SSD, etc., qual o impacto global de uma melhoria feita em apenas uma destas variáveis? É disto que trata a Lei de Amdahl.²³

Digamos assim: vamos supor simplificadaamente que um sistema, chamado Chapolin, depende essencialmente daquelas três características listadas anteriormente: clock, taxa de transferência máxima de dados da RAM e velocidade de acesso do SSD (isso é mais ou menos verdade). Aí vamos supor que cada uma destas variáveis contribui com 1/3 do impacto no processamento, ou, de outra forma, se levamos 3s para executar um programa, o processador gasta 1s, o SSD 1s e a RAM 1s, considerando operações sequenciais.

Se um engenheiro meio ruim decide investir todos os esforços em apenas uma das variáveis, qual será o ganho? Se ele *dobrar* a velocidade da RAM num sistema aperfeiçoado chamado de DidiMocó, o tempo total de processamento será 2,5s: um ganho de $0,5s/3,0s = 16,67\%$. Se ele *quadruplicar* a velocidade da RAM num terceiro sistema, o ganho em relação ao sistema inicial Chapolin será $0,75s/3,0s = 25\%$; o ganho em relação ao DidiMocó será

²³Do Gene Amdahl, arquiteto chefe do importantíssimo IBM System/360.

de $0,25s/2,5s = 10\%$ apenas. Cada vez que ele dobra a velocidade da RAM, ele obtém um ganho menor em relação ao sistema global (considerando as demais variáveis inalteradas).

No limite, se o engenheiro Mr John Super Optimizer conseguir fazer uma RAM com velocidade infinita, depois de alguns anos e muitos milhões de dólares, o sistema geral ainda vai gastar 2s para executar aquele programa inicial que nos interessava, uma melhora de 33% apenas. Portanto há um limite claro da contribuição que a otimização pode trazer, e esta é uma das consequências interessantes da Lei de Amdahl, que podemos enunciar agora:

A quantidade de aceleração que podemos obter num sistema computacional ao otimizarmos uma característica específica é proporcional ao quanto esta característica influi no sistema como um todo.

O enunciado fica um pouco confuso mas é mais ou menos assim mesmo. Já me conformei com isso.

Outra consequência interessante diz respeito à escolha do que deve ser otimizado: essencialmente, devemos nos concentrar fortemente nos gargalos de desempenho do sistema, pois aí é onde estão os maiores ganhos potenciais. Uma melhora boa no SSD, de longe o componente mais lento no sistema (veremos números quando estudarmos memórias), tem um grande impacto no sistema. Por outro lado, melhoras na velocidade de execução do processador em si ajudam para algumas tarefas, mas podem não ser tão perceptíveis no desempenho geral.

Paralelizando programas

O ponto principal para a nossa discussão são os **limites de paralelização de software**. Vamos supor que determinado programa gasta 10 segundos de CPU (ou seja, são 10 s de tempo de um processador efetivamente executando as instruções ²⁴), e também vamos supor, *com otimismo*, que 80% dele seja paralelizável. Então o tempo é dado por, supomos, algo como:

$$\begin{aligned}t_{\text{total}} &= t_{\text{inicialização}} + t_{\text{paralelizável}} + t_{\text{finalização}} \\t_{\text{total}} &= 1 + 8 + 1 = 10s\end{aligned}$$

Vamos agora dividir o trecho paralelizável em vários processadores, começando com 2 e indo para 4, 8 e 16 e vejamos o resultado:

²⁴Há definições diferentes, incluindo ou não tempo de execução do sistema operacional ou mesmo o *wall clock time*, o tempo “de relógio” para executar o programa.

$$\begin{aligned}
t_{1 \text{ uP}} &= 1 + 8 + 1 = 10s \\
t_{2 \text{ uPs}} &= 1 + \frac{8}{2} + 1 = 1 + 4 + 1 = 6s \\
t_{4 \text{ uPs}} &= 1 + \frac{8}{4} + 1 = 1 + 2 + 1 = 4s \\
t_{8 \text{ uPs}} &= 1 + \frac{8}{8} + 1 = 1 + 1 + 1 = 3s \\
t_{16 \text{ uPs}} &= 1 + \frac{8}{16} + 1 = 1 + 0,5 + 1 = 2,5s
\end{aligned}$$

É inegável a aceleração obtida com 2 processadores, que podemos definir como:

$$speedup = \frac{t_{original}}{t_{acelerado}}$$

Ou seja, temos um *speedup* de $10/6 = 1,67$ vezes. No entanto, se dobramos novamente o número de processadores, agora a aceleração relativa será de $6/4 = 1,5$, apenas. Logo estamos usando mais de uma dezena de processadores para obter uma melhora apenas marginal. (Se você quiser definições mais matemáticas, pode olhar no livro-texto do Patterson).

Uma discussão prática

Qual o ganho que podemos esperar com a paralelização? Um exemplo prático é a compilação paralela: se formos compilar um projeto grande em C com o make, temos a opção de dividir a execução em vários processadores criando, por exemplo, 8 jobs (8 processos paralelos, essencialmente) com a flag -j8. Haverá ganho? Como de hábito, a resposta é “depende.”

Usualmente os processos de compilação são “I/O bound”, ou seja, limitada pelo acesso intensivo a disco, dependendo da linguagem usada, do compilador e da localização dos arquivos-fonte. Neste caso, haverá algum ganho porque quando um processador estiver fritando ao compilar um arquivo, outro processador pode estar lendo o próximo fonte da lista. Se este tempo de fritação for curto, a melhora provavelmente não será tão significativa.

Por outro lado, a compilação pode ser “CPU bound”, ou limitada pela quantidade de processamento, embora isso não seja tão frequente. Neste caso, veremos um speedup considerável no tempo total de compilação.²⁵

Por fim, cabe observar que nesta era de processamento de quantidades massivas de dados, um dos gargalos mais notáveis, tanto no mercado de consumidores caseiros quanto de alto desempenho, é a banda de memória, ou seja, a taxa de transferência de dados pelos canais de comunicação em torno

²⁵Veja uma discussão antiga a respeito em <http://stackoverflow.com/questions/414714/compiling-with-g-using-multiple-cores>

do processador. Observe que a Intel sempre faz questão de dar números para seus barramentos principais quando lança um novo processador, e que os supercomputadores dão bastante destaque às tecnologias de barramento utilizadas na topologia escolhida.

Mais à frente iremos discutir rapidamente perfis de utilização, GPUs e NPUs e coisas similares. Se você quiser uma discussão mais científica / engenheirística sobre uso dos multicore do ponto de vista do programador e do sistema, leia sobre o modelo *roofline*, novamente lá no Patterson.

4 Uma nota sobre a história da computação

Normalmente se discute a evolução dos computadores sob a ótica dos sistemas computacionais influenciadores, desde o ENIAC, indo ao IBM System/360 e chegando ao IBM PC, quando o Microsoft Windows e Intel/AMD dominam o mercado de consumidores. Numa disciplina de arquitetura de computadores, o foco costuma ser o microprocessador, sua ISA, mas há várias alternativas interessantes.

A melhor de todas, na minha opinião, discute evolução tecnológica, mostrando contextos, inovação, causas e consequências. Temos até um programa mestrado sobre inovação na UTFPR, sem foco em computação. Historiadores são bons aqui, até porque começamos com a máquina de Anticítera, dois mil anos atrás, passamos pela computação analógica no século XX e a inevitável segunda guerra mundial, bem como as necessidades militares em tempos de paz e as vantagens de informatizar o mundo dos negócios.

Mas a história da computação também é uma história de litígios, de questões de propriedade intelectual, de concorrência desleal e de legislação norte-americana. O criador do circuito integrado entrou com processo por não ganhar nada pela invenção, os projetistas-chefes do EDVAC saíram da Universidade da Pensilvânia (e entraram com processo depois) por questões relativas a comercialização e patentes, a IBM sofreu um rigoroso processo antimonopólio pelo governo dos EUA (que diretamente influenciou na decisão de deixar aberto o padrão IBM PC, originário dos nossos computadores pessoais). Para diversão em grande volume, vá atrás de podcasts sobre a rivalidade da Intel e AMD nos anos 90.

E se você quer coisa mais técnica, mas igualmente importante, o desempenho dos processadores sempre foi limitado ou viabilizado tanto pelo estado das memórias do sistema quanto pelo resfriamento da eletrônica, que corre em paralelo com a computação valvulada e é essencial para a discussão da lei de Moore. A história das memórias é rica em bizarrices, inicialmente com capacidades minúsculas de 48 bits em memórias de linhas de atraso de mercúrio, depois com esquemas sofisticados para aumentar a velocidade em tambores de memória magnética, até finalmente chegar nas memórias semicondutoras. Há várias tecnologias antigas interessantíssimas, como a

revolucionária memória de núcleo de ferrite e a bizarra memória de tubo de raios catódicos, que era uma TV de tubo que, por assim dizer, guardava os bits como pixels na tela.